Vishwas K Singh

SME-FSD

Agenda:

- Basics of programming
- Fundamentals of python
- Variables, Data Types
  Operators
- Console IO

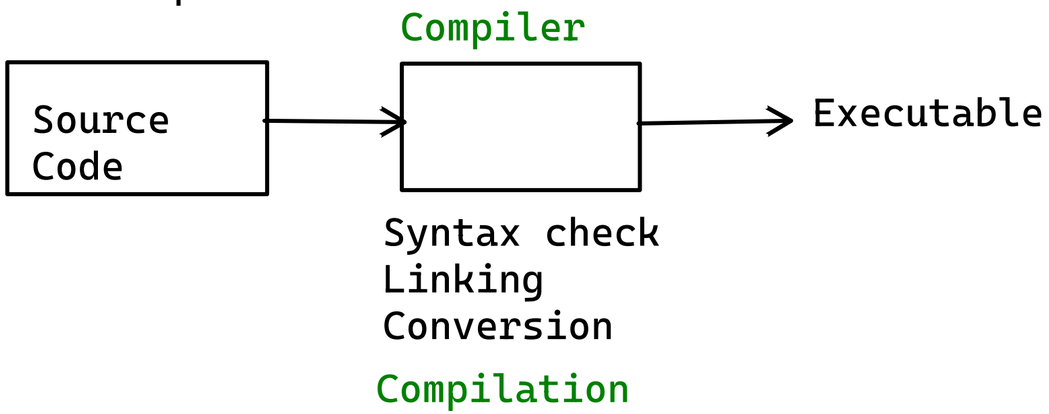
Timings:

9.30 to 11.15AM - P1
11.15 to 11.30 - break
11.30 to 1.15 -p2
1.15 to 2.15 - lunch break
2.15 to 3.45 - p3
3.45 to 4.00 - break
4.00 to 5.30 -p4

Vishwas K Singh
SME-FSD

- 1/0 - Binary Number System
- High Level Language to Machine Level language
- Two:
  1. Compilation
  2. Interpretation

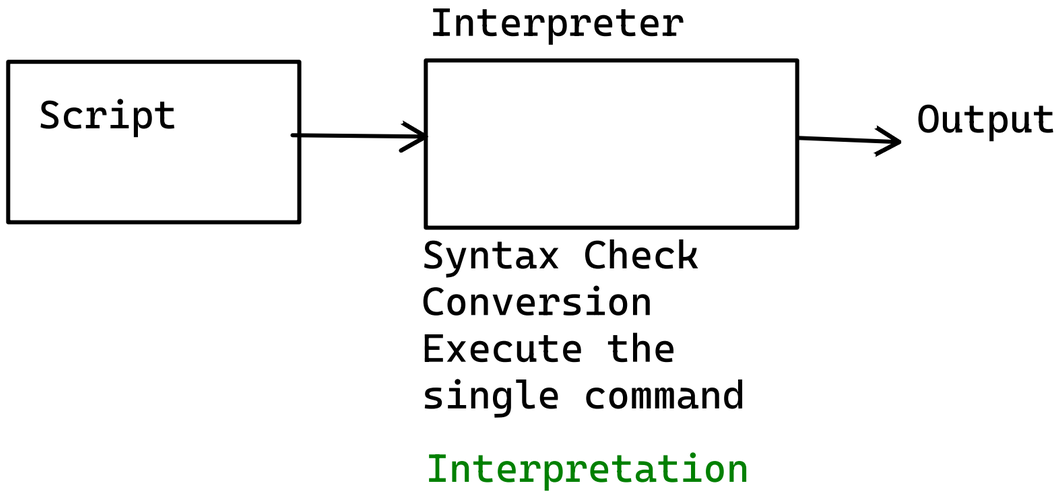
1. Compilation



C, C++, go, rust....

Pros & Cons

- No additional Software
- Secure
- Needs additional time for compilation
- granular control
- Expertise is needed
- Faster



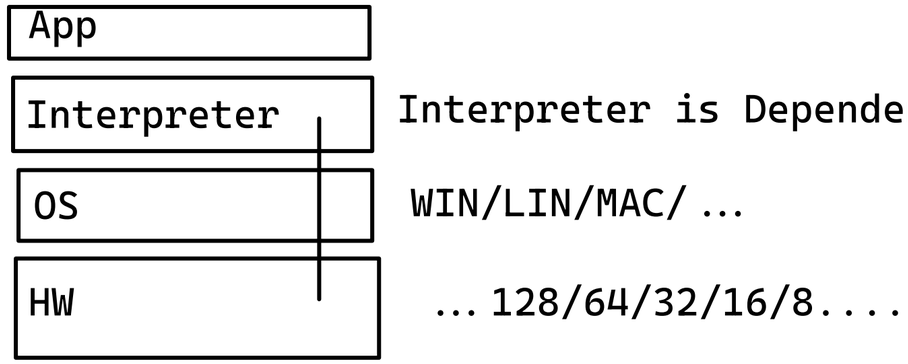
Python, JavaScript....

- Needs Interpreter
- Not secure
- No need of additional time
- less granular control more abstraction
- less faster compare to comiled langauages

Python:

- Interpreted Language
- Open Source langauge
- Platform Independent language:App can be run on any combination of HW & OS without modification
  WORA

Open Source:
Source Code is available - even for modification & Redistribution
GPL,MIT, APACHE LICENSE



Guido Van Rossum

Interpreter is Dependent on Architecture

WIN/LIN/MAC/ ...

... 128/64/32/16/8....

- General purpose language: System Apps, Web Applications, CLIs, ML/AI, DS....
- Dynamically Typed Language (later)

Vishwas K Singh
SME-FSD

Versions of Python

2.x.x
3.x.x (latest)

2.x.x ✗ 3.x.x

3.9.15 3.12.12 3.15.3

Interpreters of Python

Cython - C+Python
Jython - Java + Python
RPython - R + Python
Pypi - python interpreter written in python

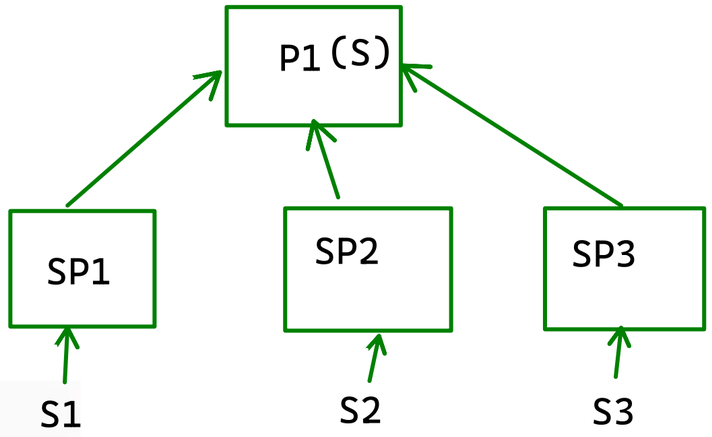
Lab: Installation of Python

How to learn a programming language
- How to store temporary data
- How to do conditions
- How to repeat
- What is the base paradigm

Base Paradigm of python
- Functional programming language

Other Paradigms:
- Object Oriented
- procedural

Functional Programming language



Example:
Scenario: You are a staff, You get CSVs or data from various dept
You have to do analysis, compile, provide a report for the
data received - HTML/PDF

Vishwas K Singh
SME-FSD

Problem:
You are a staff, You get CSVs or data from various dept
You have to do analysis, compile, provide a report for the
data received - HTML/PDF

Sub Problems:
How to read the data - read using python pandas library
Clean the data - Missing data analysis, fill with default values
How to perform analysis - Computations to be done
How to generate report - Reportlab/fpdf.... tool to generate report

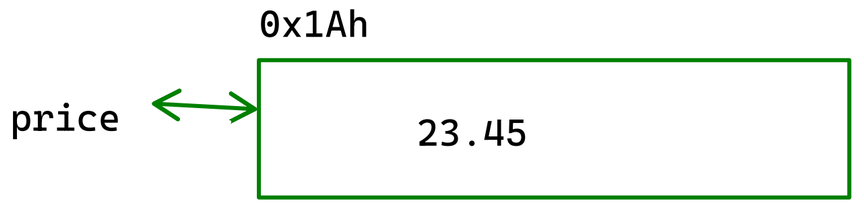
Ways of interaction with Python
- Interactive Mode
- Script Mode (.py)
- Notebook Mode/Jupyter Notebook/ Google Colab (.ipynb)
  Data Analyst / Scientist

Vishwas K Singh
SME-FSD

How to Store Data Temporarily

Variable: Place holder to data

Variable Identifier	Assigment operator	Acutal Value
price	=	23.45




```
price
print(price)
```

Rules:
- alpha_numeric or _
- Cannot start with a number
- must not be a keyword

```
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Conventions:
- Have meaningful names
- snake_case

Question: Identify Valid/ Invalid

1. vishwas
2. var
3. else1
4. def
5. !123
6. _123
7. _abc
8. abc_!

1. Valid
2. Valid
3. Valid
4. Invalid
5. Invalid
6. Valid
7. Valid
8. Invalid

```
>>> price = 23.45
>>> price
23.45
>>> print(price)
23.45
>>> type(price)
<class 'float'>
>>> age = 18
>>> age
18
>>> type(age)
<class 'int'>
>>> name = "Vishwas K Singh:
  File "<python-input-7>", line 1
    name = "Vishwas K Singh:
           ^
SyntaxError: unterminated string literal (detected at line 1)
>>> name = "Vishwas K Singh"
>>> name
'Vishwas K Singh'
>>> type(name)
<class 'str'>
>>>
```

Vishwas K Singh
SME-FSD

Data Types
- int
- float
- str
- bool
- complex

```
>>> v1 = "vishwas"
>>> type(v1)
<class 'str'>
>>> v2 = 1234
>>> type(v2)
<class 'int'>
>>> v3 = 1234.5
>>> type(v3)
<class 'float'>
>>> v4 = True
>>> type(v4)
<class 'bool'>
>>> v5 = 23+6j
>>> type(v5)
<class 'complex'>
>>>
```

```
>>> v8 = 23.45
>>> type(v8)
<class 'float'>
>>> v9 = 2345e-100
>>> v9
2.345e-97
>>> v9 = 2345e-2
>>> v9
23.45
>>> v10 = 2345e-2
>>> v10
23.45
>>> v11 = .02345e2
>>> v11
2.345
>>> v11 = .2345e2
>>> v11
23.45
>>>
```

```
>>> v6 = 12345678908765432134567890987654321234567890987654321234567890987654321234567898765432457898765432
>>> v6
12345678908765432134567890987654321234567890987654321234567890987654321234567898765432457898765432
>>> v7 = 0b1010
>>> v7
10
>>> type(v7)
<class 'int'>
>>> v7 = 0x1a
>>> v7
26
>>> type(v7)
<class 'int'>
>>> v7 = 0o1234
>>> v7
668
>>> type(v7)
<class 'int'>
>>> v7 = 30_00_000
>>> v7
3000000
>>> type(v7)
<class 'int'>
```

Vishwas K Singh
SME-FSD

Strings

```
>>> type(s1)
<class 'str'>
>>> s2 = 'vishwas'
>>> s2
'vishwas'
>>> type(s2)
<class 'str'>
>>> s3 = """Vishwas"""
>>> s3
'Vishwas'
>>> type(s3)
<class 'str'>
>>> s4 = '''Vishwas'''
>>> s4
'Vishwas'
>>> type(s4)
<class 'str'>
>>> s5 = "vishwas's book"
>>> s5
"vishwas's book"
>>> s6 = 'vishwas said "I am Vishwas, I am learning Python"'
>>> s6
'vishwas said "I am Vishwas, I am learning Python"'
>>> s7 = '''This is an example fo multiline string.
... This was created using triple quotes.
... This is a common scenario when text processing'''
>>> s7
'This is an example fo multiline string.\nThis was created using triple quotes.\nThis is a common scenario when text pro
cessing'
>>>
```

Vishwas K Singh
SME-FSD

Special Strings

```
>>> s10 = "C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace\README.md"
  File "<python-input-86>", line 1
    s10 = "C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace\README.md"
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: (unicode error) 'unicodeescape' codec can't decode bytes in position 2-3: truncated \UXXXXXXXX escape
>>> s10 = r"C:\Users\VishwasKSingh\Workspace\ey-coh6-workspace\README.md"
>>> s10
'C:\\Users\\VishwasKSingh\\Workspace\\ey-coh6-workspace\\README.md'
>>> name = "viswhas"
>>> s11 = f"This is {name}'s book"
>>> s11
"This is viswhas's book"
>>>
```

Operators

```
>>> 2+2
4
>>> 2-1
1
>>> 4*3
12
>>> 7/2
3.5
>>> 7//2
3
>>> 7%2
1
>>> 2**3
8
>>>
```

```
>>> 2 > 3
False
>>> 3 > 4
False
>>> 3 < 5
True
>>> 3 >=2
True
>>> 3<=5
True
>>> 3 !=5
True
>>> 4 == 4
True
>>>
```

```
>>> 2 > 3 and 3 < 5
False
>>> 8 != 8 and 5 < 6
False
```

- Non Zero value is always true
- 0 is always false

2 and 3	2 or 4	0 or 9
2 and 5	4 or 2	9 or 0
5 and 2	9 or 2	0 and 9
5 and 3	2 or 9	9 and 0

-1 and 5
5 and -1

Vishwas K Singh
SME-FSD

Short circuit evaulation
- And Condition
expr1 and expr2
expr2 is evaluated only when expr1 is true
- Or Condition
expr1 or expr2
expr2 is not evaluated at all if expr1 is true

```
>>> x = 10
>>> y = 2
>>> x /y
5.0
>>> y = 0
>>> x/y
Traceback (most recent call last):
  File "<python-input-124>", line 1, in <module>
    x/y
    ~^~
ZeroDivisionError: division by zero
>>> y != 0 and x / y
False
>>> y = 2
>>> y != 0 and x / y
5.0
>>>
```

PEDMAS

```
23 / 6 * 4 + 3 ** (2 * 2)
```

Console I/O
- I can take a value from the user using console
- input(prompt) → is a function to do that
- it always returns a str
- you have to convert it explicitly

```
>>> print("Vishwas","k","singh")
Vishwas k singh
>>> print("Vishwas","k","singh", sep="*")
Vishwas*k*singh
>>> print("Vishwas","k","singh", sep="*", end="---")
>>> was*k*singh---
```

Vishwas K Singh
SME-FSD

1. Simple Interest Formula: si = (p*t*r)/100

2. SIP vs Lumpsum calculator

3.

Users	City
Vishwas	Pune
Raju	Goa
Sachin	Delhi

4. Brokerage Calculator

5. Income Tax Calculator

Vishwas K Singh

SME-FSD

Day 1 Coverage
- Basics of programming
- Fundamentals of python
- Variables, Data Types Operators
- Console IO

What to expect
- Conditionals
- Loops
- Str
- List, Dict, Tuple Set

Vishwas K Singh
SME-FSD

Day2 Agenda

- Conditionals
- Loops
- DS

Vishwas K Singh
SME-FSD

Conditionals

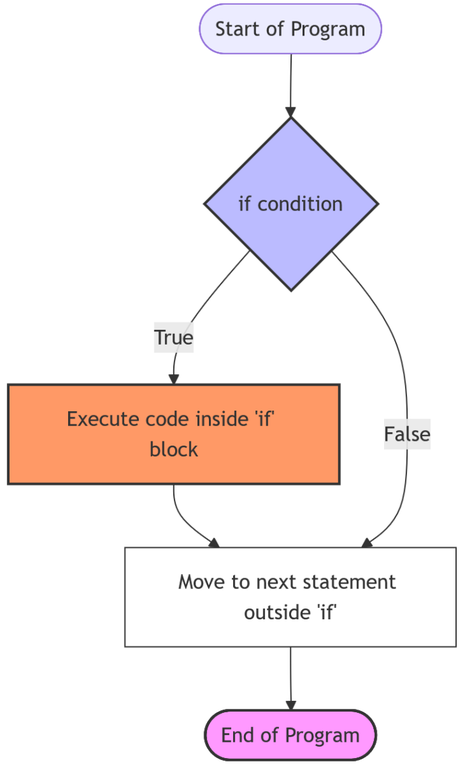
if
if-else
if-elif-else
nested-if

Block of Code

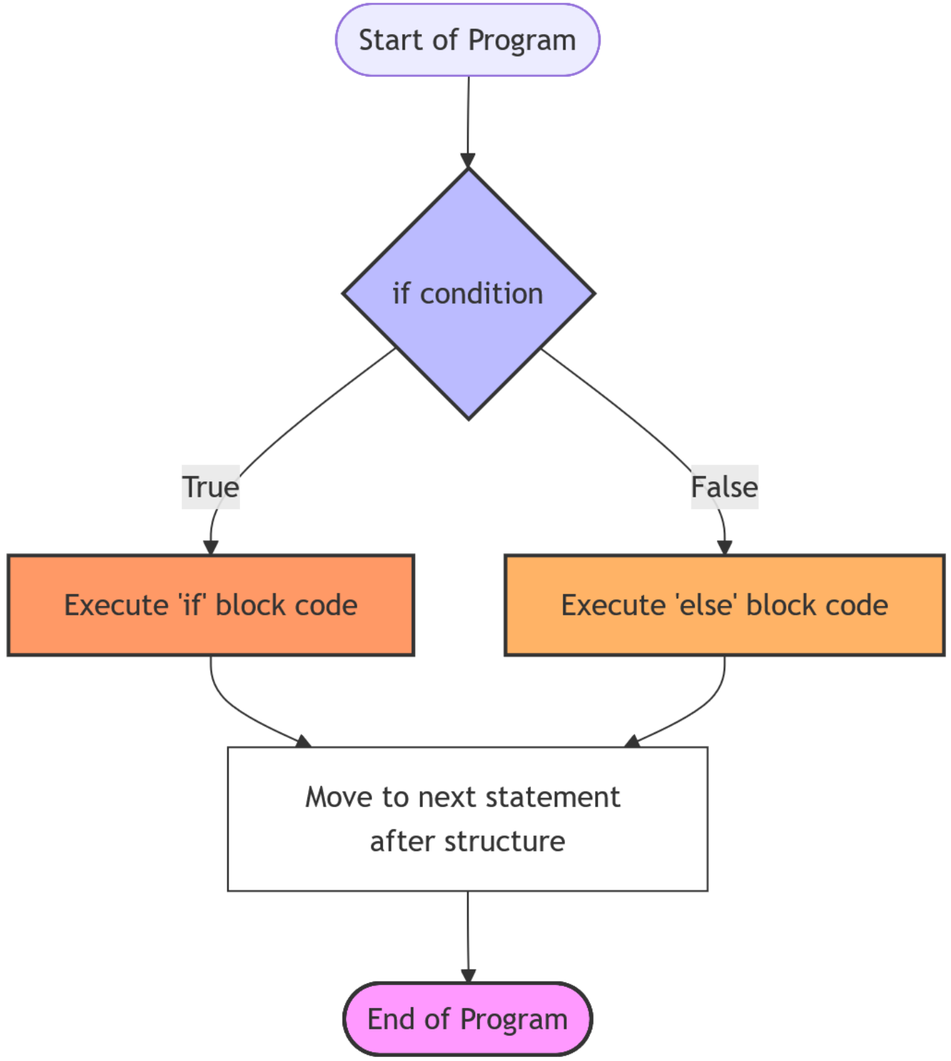
————————:
    s1
    s2
    s3
sn1
sn2

```
>>> x = 10
>>> if x > 10:
...     print("x is greater than 10")
...
>>> if x < 10:
...     print("x is less than 10")
...
>>> if x <= 10:
...     print("x is less than or equal to 10")
...
x is less than or equal to 10
>>>
```

```
if condn:
    s1
    s2
    s3
sn1
sn2
```



```
if condn:
   s1
   s2
else:
   s3
   s4
sn1
sn2
```

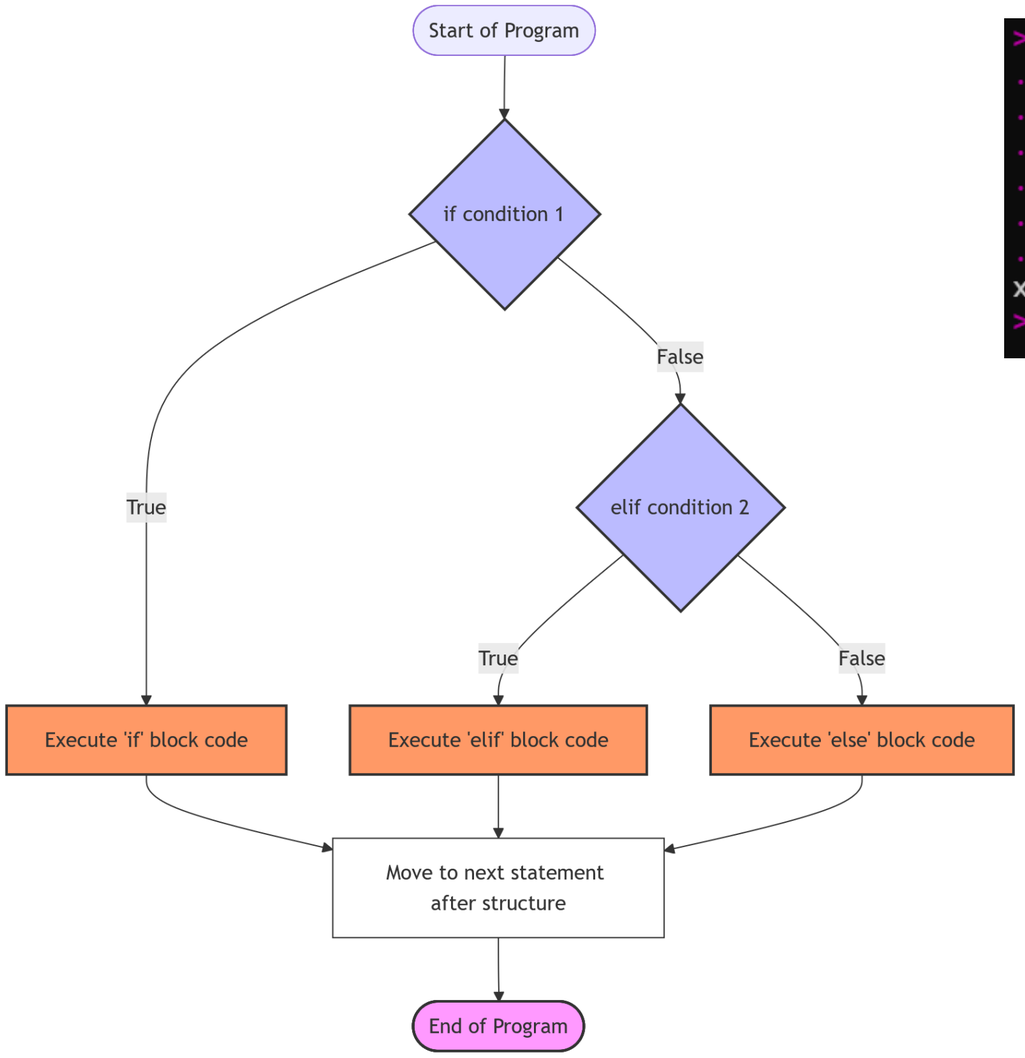



```
>>> x = 10
>>> if x == 10:
...     print("x is equal to 10")
... else:
...     print("x is not equal to 10")
...
x is equal to 10
>>>
```

Vishwas K Singh
SME-FSD

if -elif-else

```
if condn1:
    s1
    s2
elif condn2:
    s3
    s4
else:
    s5
    s6
sn1
sn2
```




```
>>> if x >=56:
...     print("x is greater than or equal to 56")
... elif x <= 46:
...     print("x is less than or equal to 46")
... else:
...     print("x is between 46 & 56")
...
x is greater than or equal to 56
>>>
```

Vishwas K Singh
SME-FSD

What will be printed

```
x = 50
if x > 40:
   print("x>40")
elif x > 45:
   print("x>45")
elif x > 47:
   print("x > 47")
else:
   print("x < 100")
```

A.
```
print("x>40")
print("x>45")
print("x > 47")
print("x < 100")
```

B.
```
print("x>45")
print("x > 47")
print("x < 100")
```

C.
```
print("x < 100")
```

D.
```
print("x>40")
```

E.
```
print("x > 47")
```

Lab: Calculate Income tax based on the tax slab

ANNUAL INCOME	WILL BE TAXED AT...
Up to ₹2.5 lakh	Nil
₹2.5 lakh to ₹5 lakh	5%
₹5 lakh to ₹7.5 lakh	10%
₹7.5 lakh to ₹10 lakh	15%
₹10 lakh to ₹12.5 lakh	20%
₹12.5 lakh to ₹15 lakh	25%
Above ₹15 lakh	30%

Vishwas K Singh
SME-FSD

Loops

Repeat block of statements

For Loops

```
for i in range(n):
    print(i)
```

```
range(stop) -> range object
range(start, stop[, step]) -> range object

Return an object that produces a sequence of integers from start (inclusive)
to stop (exclusive) by step.  range(i, j) produces i, i+1, i+2, ..., j-1.
start defaults to 0, and stop is omitted!  range(4) produces 0, 1, 2, 3.
These are exactly the valid indices for a list of 4 elements.
When step is given, it specifies the increment (or decrement).
```

```
>>> for i in range(11):
...     print(i)
...
0
1
2
3
4
5
6
7
8
9
10
>>>
>>> for i in range(1,11):
...     print(i)
...
1
2
3
4
5
6
7
8
9
10
>>>
```

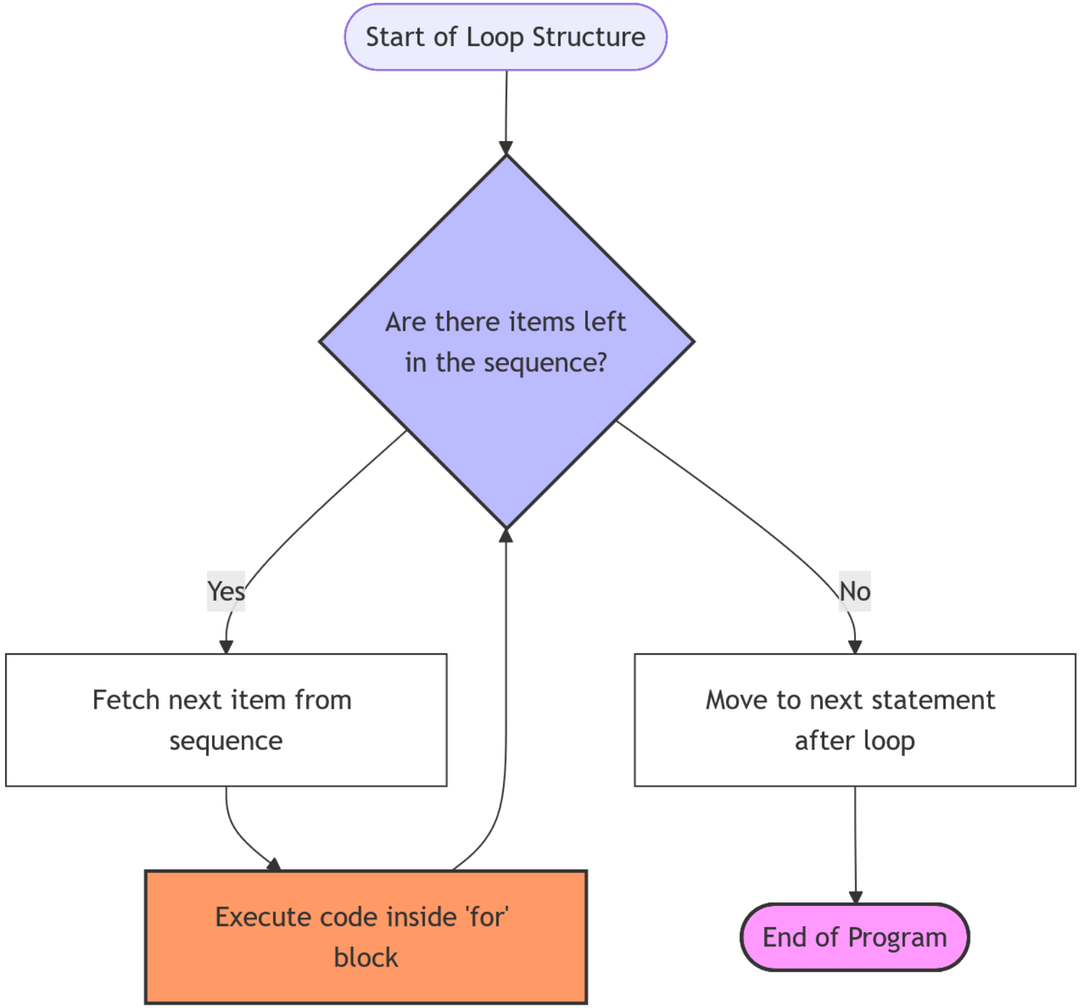
Vishwas K Singh
SME-FSD

While Loops

```
x = 10

while x > 0:
    print(x)
    x -= 1
```

```
>>> x = 10
>>> while x > 0:
...     print(x)
...     x -= 1
...
10
9
8
7
6
5
4
3
2
1
>>>
```



Vishwas K Singh

SME-FSD

Strings
- Immutable (cant be modified)
- Text Processing
- Indexes
- Sequence of Characters
- Slicing

- Always from left to right
EXCEPT WHEN STEP IS -VE

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
V	I	S	H	W	A	S		K		S	I	N	G	H
-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
>>> S1 = "VISHWAS K SINGH"
>>> len(S1)
15
>>> S1[1]
'I'
>>> S1[2]
'S'
>>> S1[14]
'H'
>>> S1[-1]
'H'
>>> S1[-2]
'G'
>>> S1[-11]
'W'
>>> S1[-15]
'V'
```

```
>>> S1[0:7]
'VISHWAS'
>>> S1[0:15]
'VISHWAS K SINGH'
>>> S1[10:15]
'SINGH'
>>> S1[10:14]
'SING'
>>> S1[10:200]
'SINGH'
>>> S1[-15:-8]
'VISHWAS'
>>> S1[-5:-2]
'SIN'
>>> S1[-5:0]
''
>>> S1[-5:-1]
'SING'
>>> S1[-5:]
'SINGH'
>>> S1[:-8]
'VISHWAS'
```

```
>>> S1[0:-8]
'VISHWAS'
>>> S1[-15:7]
'VISHWAS'
>>> S1[10:]
'SINGH'
>>> S1[-5:]
'SINGH'
>>> S1[10:-2]
'SIN'
>>> S1[-5:13]
'SIN'
```

```
>>> S1[:]
'VISHWAS K SINGH'
>>> S1[::2]
'VSWSKSNH'
>>> S1[::3]
'VHS N'
```

```
>>> S1[::-1]
'HGNIS K SAWHSIV'
>>> S1[::-2]
'HNSKSWSV'
>>> S1[::-3]
'HIKAS'
>>>
```

```
>>> S1[6:3:-1]
'SAW'
>>> S1[-9:-12:-1]
'SAW'
>>> S1[6:-12:-1]
'SAW'
>>> S1[-9:3:-1]
'SAW'
```

Vishwas K Singh
SME-FSD

String Methods

```
>>> dir(str)
['__add__', '__class__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__', '__getat
tribute__', '__getitem__', '__getnewargs__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__', '__i
ter__', '__le__', '__len__', '__lt__', '__mod__', '__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__', '__repr
__', '__rmod__', '__rmul__', '__setattr__', '__sizeof__', '__str__', '__subclasshook__', 'capitalize', 'casefold', 'cent
er', 'count', 'encode', 'endswith', 'expandtabs', 'find', 'format', 'format_map', 'index', 'isalnum', 'isalpha', 'isasci
i', 'isdecimal', 'isdigit', 'isidentifier', 'islower', 'isnumeric', 'isprintable', 'isspace', 'istitle', 'isupper', 'joi
n', 'ljust', 'lower', 'lstrip', 'maketrans', 'partition', 'removeprefix', 'removesuffix', 'replace', 'rfind', 'rindex',
'rjust', 'rpartition', 'rsplit', 'rstrip', 'split', 'splitlines', 'startswith', 'strip', 'swapcase', 'title', 'translate
', 'upper', 'zfill']
```

```
>>> S1
'VISHWAS K SINGH'
>>> S1.lower()
'vishwas k singh'
>>> S1.capitalize()
'Vishwas k singh'
```

```
>>> S1.center(80, "-")
'------------------------------VISHWAS K SINGH-------------------------------'
>>> S1.rjust(80, "-")
'----------------------------------------------------------------VISHWAS K SINGH'
>>> S1.ljust(80, "-")
'VISHWAS K SINGH----------------------------------------------------------------'
>>> S1.swapcase()
'vishwas k singh'
>>> S1.replace("SINGH", "*****")
'VISHWAS K *****'
>>> S1.startswith("V")
True
```

Vishwas K Singh
SME-FSD

Lab:

1. Username extracter:
   email: vishwas@cloudthat.com
   username: vishwas
2. valid email:
   email: vishwas@cloudthat.com
   Vaild
   email: vishwas@cloudthat@.com
   Invalid
   email: vishwas
   Invalid
3. Valid IPv4 Address
   ip: 192.168.1.1
   Valid
   ip: a.b.192.1
   Invalid
   ip: 279.172.2.1
   Invalid
4. private / public ipv4

Ipv4:
- 4 Fields
- Dotted Decimal Notation
- 0 to 255

RFC 1918 name	IP address range
24-bit block	10.0.0.0 – 10.255.255.255
20-bit block	172.16.0.0 – 172.31.255.255
16-bit block	192.168.0.0 – 192.168.255.255

Vishwas K Singh
SME-FSD

List
- Stores multiple values, duplicates also
- it is mutable
- it is heterogeneous
- indexing, slicing

```
>>> l1 = []
>>> l2 = list()
>>> l3 = ["apple","Mango",1,2,34,{1:"Vishwas"},("Pune","Goa")]
>>> l3[0]
'apple'
>>> l3[1]
'Mango'
>>> dir(list)
['__add__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq_
_', '__format__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__iadd__', '__imul
__', '__init__', '__init_subclass__', '__iter__', '__le__', '__len__', '__lt__', '__mul__', '__ne__', '__new__', '__redu
ce__', '__reduce_ex__', '__repr__', '__reversed__', '__rmul__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '
__subclasshook__', 'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove', 'reverse', 'sort']
>>>
```

Vishwas K Singh
SME-FSD

Dict
- Key Value
- Key must be unique
- Heterogeneous
- Key can be any data type, int & str recommended

```
>>> d1 = {}
>>> type(d1)
<class 'dict'>
>>> d2 = dict()
>>> type(d2)
<class 'dict'>
>>> d3 = {"user":"vishwas","city":"pune","company":"ct",1:"Apple",2:"mango"}
>>> d3
{'user': 'vishwas', 'city': 'pune', 'company': 'ct', 1: 'Apple', 2: 'mango'}
>>> d3["user"]
'vishwas'
>>> d3["city"]
'pune'
>>> d3[5]="banana"
```

```
>>> dir(dict)
['__class__', '__class_getitem__', '__contains__', '__delattr__', '__delitem__', '__dir__', '__doc__', '__eq__', '__form
at__', '__ge__', '__getattribute__', '__getitem__', '__getstate__', '__gt__', '__hash__', '__init__', '__init_subclass__
', '__ior__', '__iter__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__reduce__', '__reduce_ex__', '
__repr__', '__reversed__', '__ror__', '__setattr__', '__setitem__', '__sizeof__', '__str__', '__subclasshook__', 'clear'
, 'copy', 'fromkeys', 'get', 'items', 'keys', 'pop', 'popitem', 'setdefault', 'update', 'values']
>>>
```

Vishwas K Singh
SME-FSD

Lab: Given a list of ip_address remove invalid ip addresses without filter function
["123.1.25.1","72.24.1.6","a.b","192.168","192.168.1.1"]

Lab: Frequency Counter
"vishwasksingh"
"v": 1, "i":2, "s":3,"h":2.....

Vishwas K Singh
SME-FSD

Tuple
- Immutable
- acts like a list - indexed, sliced, heterogeneous
- Python uses this in backend for passing the data

```
>>> t1 = ("Apple","Mango","grapes",1,2,3,"grapes",{1:"vishwas",2:"arjun"})
>>> t1[0]
'Apple'
>>> t1[-1]
{1: 'vishwas', 2: 'arjun'}
>>> t1[:4]
('Apple', 'Mango', 'grapes', 1)
>>>
```

```
>>> t2 = "pune","mumbai","goa","delhi"
>>> t2
('pune', 'mumbai', 'goa', 'delhi')
>>> a,b,c,d = t2
>>> a
'pune'
>>> b
'mumbai'
>>> c
'goa'
>>> d
'delhi'
>>>
```

```
>>> t3 = (1,2,3,4,5)
>>> t4 = (1,2,3,3,7)
>>> t3>t4
True
>>> t3<t4
False
>>> t3.index(4)
3
>>> t4.count(3)
2
>>>
```

Vishwas K Singh
SME-FSD

Set

- Distinct, homogeneous

```
>>> s1 = {45,67,89,1,2,3,4,45,6,7,89}
>>> s1
{1, 2, 3, 67, 4, 6, 7, 45, 89}
>>> type(s1)
<class 'set'>
>>> s2 = s1.copy()
>>> s2
{1, 2, 3, 67, 4, 6, 7, 45, 89}
>>> s1.clear()
>>> s1
set()
>>> s1.add("vishwas"
... )
>>> s1
{'vishwas'}
>>> s1.add("arjun")
>>> s1
{'arjun', 'vishwas'}
>>> dir(set)
['__and__', '__class__', '__class_getitem__', '__contains__', '__delattr__', '__dir__', '__doc__', '__eq__', '__format__
', '__ge__', '__getattribute__', '__getstate__', '__gt__', '__hash__', '__iand__', '__init__', '__init_subclass__', '__i
or__', '__isub__', '__iter__', '__ixor__', '__le__', '__len__', '__lt__', '__ne__', '__new__', '__or__', '__rand__', '__
reduce__', '__reduce_ex__', '__repr__', '__ror__', '__rsub__', '__rxor__', '__setattr__', '__sizeof__', '__str__', '__su
b__', '__subclasshook__', '__xor__', 'add', 'clear', 'copy', 'difference', 'difference_update', 'discard', 'intersection
', 'intersection_update', 'isdisjoint', 'issubset', 'issuperset', 'pop', 'remove', 'symmetric_difference', 'symmetric_di
fference_update', 'union', 'update']
>>>
```

Vishwas K Singh
SME-FSD

Lab:
1. Given a list which contains duplicates, create a list with dictinct items
2. Given two strings
   s1 = "abbadda", s2= "abcbedf"
   output 2 strings where op1 will contain characters in s1 which are not in s2
 op2 will contain characters in s2 which are not in s1

Lab: Given a log data as a list of strings. Which contains data about login attempts by a email id. based on the number of login attempts, find out the email ids which may be marked as suspicious.

Assumption: Number of login attempts > 2 (mark it as suspicious)

Vishwas K Singh
SME-FSD

Day 2 Summary
- Conditionals
- Loops
- DS

What to expect tomorrow:
- Functions
- Modules
- Intermediate concepts: comprehensions, iterators, generators
- oops
- file handling & exceptions